

04-JVM

JVM 面试题

1. 内存结构

问：JVM 内存结构？

答：

线程私有：

├ 程序计数器 (PC Register)

├ 虚拟机栈 (VM Stack)

└ 本地方法栈 (Native Stack)

└ 当前字节码行号

└ 局部变量表、操作数栈、方法出口

└ Native 方法

线程共享：

├ 堆 (Heap)

├ 方法区 / 元空间 (Metaspace)

└ 对象实例，GC 主要区域

└ 类元信息、常量、静态变量

├ 新生代 (Eden + Survivor0 + Survivor1)

└ 老年代

JDK 8 变化：永久代 (PermGen) 移除，元空间使用本地内存。

常用参数：-Xms 初始堆、-Xmx 最大堆、-Xss 栈大小、-XX:MetaspaceSize。

问：栈中存的是指针还是对象？

答：栈存引用 (reference)，对象实例在堆中。

```
User user = new User(); // user 是栈上的引用, User 对象在堆上
```

局部变量表存基本类型值或对象引用；通过引用访问堆中对象。

问：堆分为哪几部分？大对象一般放哪？

答：

区域	说明
新生代	Eden + Survivor0 + Survivor1 (默认 8:1:1)
老年代	长期存活对象、大对象

大对象：超过 -XX:PretenureSizeThreshold (G1 为 Region 一半) 可能直接进入老年代，避免 Eden 间大量复制。

G1 Humongous Region：超大对象占用连续 Humongous Region。

问：程序计数器的作用？为什么私有？

答：- 作用：记录当前线程执行的字节码 行号/地址，分支、循环、异常跳转后恢复执行位置 - 私有原因：各线程独立执行，需要各自的行号指针 - 唯一不会 OOM 的区域

问：方法区存什么？String 存在哪？

答：方法区 / 元空间存储：- 类信息（字段、方法、字节码）- 运行时常量池 - 静态变量（JDK 7+ 静态变量在堆，引用指向堆对象）- JIT 编译后的代码缓存

String：- 字面量 → 字符串常量池（JDK 7+ 在 堆 中）- new String("abc") → 堆中新建对象

问：堆和栈的区别？

答：

对比	堆	栈
存储	对象实例、数组	局部变量、方法调用
线程	共享	私有
GC	是	否
异常	OutOfMemoryError	StackOverflowError
速度	较慢	快

2. 类加载

问：类加载过程？

答：

加载 → 验证 → 准备 → 解析 → 初始化 → 使用 → 卸载

阶段	说明
加载	读取 .class，生成 Class 对象
验证	文件格式、元数据、字节码、符号引用
准备	静态变量分配内存并设零值
解析	符号引用转直接引用
初始化	执行 <clinit>()，真正赋值

问：什么是双亲委派？为什么要用？

答：类加载器收到请求时，先委派父加载器加载，父加载不了才自己加载。

Bootstrap → Extension → Application → 自定义

好处： 1. 避免类重复加载 2. 保护核心类（如 `java.lang.String` 不会被自定义类替换）

破坏场景：SPI（JDBC）、Tomcat 热部署、OSGi。

问：有哪些类加载器？什么时候会触发类初始化？

答：

类加载器	加载范围
Bootstrap	lib/rt.jar 等核心类（C++ 实现）
Extension	lib/ext
Application	classpath 用户类
自定义	继承 ClassLoader

触发初始化的 6 种情况（主动引用）： 1. new / 静态字段读写 / 静态方法调用 2. 反射调用 3. 初始化子类时先初始化父类 4. 启动 main 方法所在类 5. `MethodHandle` 解析 `REF_getStatic` 等 6. 实现了 `Initialization` 的接口

不会初始化：`Class.forName(..., false)`、子类引用父类静态字段（只初始化父类）。

3. 垃圾回收

问：如何判断对象可以被回收？

答：

引用计数法（Java 未采用）：引用为 0 即回收，无法解决循环引用。

可达性分析（主流）：从 GC Roots 出发，不可达的对象可回收。

GC Roots 包括： – 虚拟机栈中引用的对象 – 方法区静态属性引用的对象 – 方法区常量引用的对象 – 本地方法栈 JNI 引用的对象 – 被 `synchronized` 持有的对象 – JVM 内部引用（基本类型 `Class`、系统类加载器等）

问：垃圾回收算法有哪些？

答：

算法	过程	优点	缺点
标记-清除	标记 → 清除	简单	内存碎片
标记-整理	标记 → 移动存活对象 → 清除边界外	无碎片	移动开销大
复制	存活对象复制到另一块	无碎片、快	浪费一半空间
分代收集	新生代复制、老年代标记-整理/清除	综合最优	实现复杂

新生代：Eden : Survivor = 8:1:1，Minor GC 后存活对象复制到 Survivor，年龄达阈值（默认 15）进入老年代。

问：有哪些垃圾收集器？

答：

收集器	区域	特点
Serial	新生代	单线程，STW
ParNew	新生代	多线程 Serial（已 deprecated）
Parallel Scavenge	新生代	吞吐量优先
Serial Old	老年代	单线程标记-整理
Parallel Old	老年代	多线程标记-整理
CMS	老年代	低延迟，已移除（JDK 14）
G1	全堆	JDK 9+ 默认，Region 划分
ZGC	全堆	超低延迟（<1ms），TB 级堆
Shenandoah	全堆	类似 ZGC

JDK 21 主流：G1（默认）、ZGC、Parallel。

问：CMS 和 G1 的区别？如何选型？

答：

对比	CMS	G1
区域	传统分代	Region 划分
算法	标记-清除（有碎片）	标记-整理 + 复制
停顿	追求低延迟	-XX:MaxGCPauseMillis 可控
状态	JDK 14 移除	JDK 9+ 默认

选型：

场景	推荐
通用 Web 服务	G1
超大堆 + 极低延迟	ZGC (JDK 21 分代)
批处理、吞吐优先	Parallel
堆 < 4G	也可 Parallel

问：GC 只回收堆吗？

答： 主要回收堆，但也会回收： – 方法区/元空间：Class Unloading（类加载器不可达时卸载类） – 直接内存：通过虚引用 PhantomReference 关联 Cleaner 回收 DirectByteBuffer
栈、程序计数器由线程生命周期管理，不由 GC 回收。

问：Minor GC、Major GC、Full GC 区别？

答： – Minor GC：新生代 GC，频繁、速度快 – Major GC：老年代 GC – Full GC：整堆 + 方法区，STW 长，应尽量避免

触发 Full GC： – 老年代空间不足 – 元空间不足 – `System.gc()`（建议） – CMS GC 失败

问：哪些阶段会 Stop The World？

答： GC 中需要暂停用户线程的阶段： – 初始标记（G1/CMS） – 再标记 – 清理/转移（复制存活对象） – Young GC / Full GC 整体

G1 中耗时最长通常是 **转移阶段**（复制存活对象）。

4. OOM 与排查

问：常见 OOM 类型？

答：

类型	原因
Java heap space	堆内存不足，对象太多
StackOverflowError	栈深度超限（递归过深）
Metaspace	类元信息过多
Direct buffer memory	直接内存（NIO）不足
unable to create new native thread	线程数超过 OS 限制

问：内存泄漏和内存溢出有什么区别？

答：

对比	内存泄漏	内存溢出 (OOM)
本质	无用对象仍被引用，GC 无法回收	需要内存 > 可用内存
表现	内存缓慢增长，最终可能 OOM	直接抛 OutOfMemoryError
常见原因	静态集合、ThreadLocal、未关闭连接、监听器未移除	大对象、泄漏累积、堆配置过小

排查泄漏：-XX:+HeapDumpOnOutOfMemoryError → MAT 看 Dominator Tree → 追踪 GC Roots 引用链。

问：JVM 排查工具有哪些？

答：

工具	用途
jps	查看 Java 进程
jstat	GC 统计 (jstat -gcutil)
jmap	堆 dump (jmap -dump)
jstack	线程栈，查死锁
MAT	分析 heap dump
VisualVM / Arthas	综合诊断

排查思路：看日志 → jstack 看线程 → jmap dump → MAT 分析大对象和 GC Roots 引用链。

5. 运行时数据区详解

问：JVM 运行时数据区各区域存储什么？哪些会 OOM？

答：

区域	存储内容	OOM
程序计数器	当前线程执行的字节码行号	不会
虚拟机栈	栈帧（局部变量表、操作数栈、动态链接、方法出口）	StackOverflowError / OOM
本地方法栈	Native 方法栈帧	同栈
堆	对象实例、数组	heap space OOM
元空间	类元信息、常量池、静态变量 (JDK8+)	Metaspace OOM
直接内存	NIO DirectByteBuffer	Direct buffer OOM

JDK 7 变化：字符串常量池从方法区移到堆；JDK 8 永久代移除，元空间用本地内存。

问：虚拟机栈帧结构详解？

答：每个方法调用创建一个栈帧，包含：1. **局部变量表**：存方法参数和局部变量（Slot 槽，基本类型占 1 槽，long/double 占 2 槽）2. **操作数栈**：计算过程中的临时数据（如 i++ 的读-改-写）3. **动态链接**：指向运行时常量池的方法引用4. **方法返回地址**：方法正常/异常退出的返回位置

StackOverflowError：递归过深，栈帧过多。**OOM**：栈可动态扩展，扩展失败时 OOM。

6. 对象创建过程

问：对象创建的完整过程？

答：

1. 类加载检查（类是否已加载、解析、初始化）
2. 分配内存
 - ├ 指针碰撞（堆规整，Serial/ParNew + CMS）
 - └ 空闲列表（堆不规整，Mark-Sweep）
3. 初始化零值（保证字段有默认值）
4. 设置对象头（Mark Word + 类型指针）
5. 执行 <init> 构造方法

对象内存布局：– **对象头**：Mark Word（hash、GC 年龄、锁状态）+ 类型指针（Klass Pointer，压缩指针时 4 字节）– **实例数据**：字段（对齐填充）– **对齐填充**：8 字节对齐

问：对象头 Mark Word 存储什么？

答：32 位 JVM 占 4 字节，64 位占 8 字节，存储：– 哈希码（identity hashCode）– GC 分代年龄（4 bit，最大 15）– 锁状态标志（无锁/偏向/轻量/重量）– 线程 ID（偏向锁）– 偏向时间戳

锁升级信息就存在 Mark Word 中，无需额外数据结构。

7. 逃逸分析与 TLAB

问：什么是逃逸分析？JVM 做了哪些优化？

答：JIT 分析对象 **作用域是否逃出方法或线程**：– **方法逃逸**：对象被外部方法引用 – **线程逃逸**：对象被其他线程访问

未逃逸优化：1. **栈上分配**：对象在栈上分配，方法结束自动销毁，无需 GC 2. **标量替换**：对象拆散为基本类型，存寄存器/栈 3. **锁消除**：未逃逸的对象加锁会被消除（如 StringBuffer 局部变量）

开启：`-XX:+DoEscapeAnalysis`（默认开启）

问：什么是 TLAB？为什么需要它？

答：Thread Local Allocation Buffer，每个线程在 Eden 区预分配一小块私有内存。

作用：– 对象分配是高频操作，TLAB 避免多线程 CAS 竞争 Eden 区指针 – 分配只需在自己的 TLAB 内移动指针（Bump-the-Pointer），极快

流程：先在 TLAB 分配 → TLAB 不够 → 新申请 TLAB 或直接在 Eden 分配（需 CAS）。

参数：-XX:+UseTLAB（默认开启）、-XX:TLABSize。

8. Full GC 触发条件

问：Full GC 有哪些触发条件？

答：1. 老年代空间不足：大对象直接进入老年代、长期存活对象晋升 2. 元空间不足：加载类过多，-XX:MetaspaceSize 不够 3. System.gc()：建议 JVM 执行 Full GC（DisableExplicitGC 可禁用） 4. CMS GC 失败：Concurrent Mode Failure，晋升过快，CMS 来不及回收 5. G1 Evacuation Failure：回收时找不到足够 Region 存放存活对象 6. 空间分配担保失败：Minor GC 前检查老年代最大连续空间 < 新生代所有对象大小 7. Promotion Failed：Survivor 放不下，晋升老年代也失败

危害：STW 时间长（秒级），接口超时、吞吐量骤降。

问：如何减少 Full GC？

答：1. 合理设置堆大小（-Xms = -Xmx 避免动态扩展） 2. 增大新生代比例，减少对象过早晋升 3. 排查内存泄漏（MAT 分析 dump） 4. 避免大对象（超过 -XX:PretenureSizeThreshold 直接进老年代） 5. 选用 G1/ZGC 等低延迟收集器 6. 禁用 System.gc()（-XX:+DisableExplicitGC）

9. ZGC 与分代

问：ZGC 的特点和工作原理？

答：目标：停顿时间 < 1ms，支持 TB 级堆。

核心技术：– 着色指针（Colored Pointer）：64 位指针中借用位标记对象状态（Marked0/1、Remapped、Finalizable） – 读屏障（Load Barrier）：读引用时检查并修正指针，保证并发移动对象时读到正确地址 – 并发转移：标记和移动对象几乎全程并发，STW 极短

阶段：Pause Mark Start → Concurrent Mark → Pause Mark End → Concurrent Prepare Relocate → Pause Relocate Start → Concurrent Relocate

问：ZGC 分代（Generational ZGC）是什么？JDK 21 有什么变化？

答：JDK 21 之前 ZGC 不分代，所有对象同等对待，Young GC 效率不如 G1。

JDK 21 正式支持分代 ZGC（JEP 439）： – 区分 Young Generation 和 Old Generation – 新生代用复制算法，频繁 Minor GC 回收短生命周期对象 – 老年代并发回收，保持低延迟

效果：吞吐提升 10%+，分配速率更高，GC 开销降低。

启用：`-XX:+UseZGC -XX:+ZGenerational`（JDK 21+ 默认分代）

10. G1 补充

问：G1 收集器 Region 和 Mixed GC 是什么？

答：G1 将堆划分为多个 Region（1~32MB），每个 Region 可以是 Eden/Survivor/Old/Humongous。

回收过程：1. Young GC：回收所有 Eden + Survivor 2. 并发标记：标记存活对象 3. Mixed GC：回收所有 Young + 部分 Old Region（根据停顿目标 `-XX:MaxGCPauseMillis` 选择）

Humongous Region：存放大对象（超过 Region 一半），直接进老年代逻辑。

11. 排查案例

问：线上接口突然变慢，如何排查 JVM 问题？

答：步骤：1. 看监控：CPU、内存、GC 频率/耗时（Prometheus + Grafana）2. `jstat -gcutil <pid> 1000`：观察 GC 是否频繁，Full GC 是否增多 3. `jstack <pid>`：看是否有大量 BLOCKED 线程、死锁 4. Arthas：`dashboard`、`thread -n 3`（最忙线程）、`trace` 慢方法 5. 若怀疑内存：`jmap -dump:live,format=b,file=heap.hprof <pid>` 6. MAT 分析：Dominator Tree 找大对象、GC Roots 引用链找泄漏

问：一个典型的内存泄漏排查案例？

答：现象：服务运行数天后 OOM，重启恢复。

排查：1. dump 堆：`jmap -dump:live,file=heap.hprof <pid>` 2. MAT 打开，Histogram 按 Retained Heap 排序 3. 发现 `byte[]` 占用 2GB，Dominator Tree 追踪到 `ConcurrentHashMap` 4. 引用链：`ThreadLocalMap` → `ThreadLocal` → 大 Map 缓存 5. 根因：ThreadLocal 存了大 Map，线程池线程不销毁，Map 只增不减 6. 修复：改用 Caffeine 带过期、finally 中 `threadLocal.remove()`

问：CPU 100% 如何排查？

答：1. `top -Hp <pid>` 找 CPU 最高的线程 TID 2. `printf '%x\n' <TID>` 转 16 进制 3. `jstack <pid> | grep <hex_tid> -A 30` 看线程栈 4. 常见原因：死循环、频繁 GC（GC 线程 CPU 高）、正则回溯、序列化

Arthas 快捷：`thread -n 3` 直接列最忙 3 个线程及栈。

12. 其他深入

问：强软弱虚引用分别是什么？应用场景？

答：

引用	回收时机	场景
强引用	永不回收（OOM 前）	普通 new
软引用 SoftReference	内存不足时回收	图片缓存
弱引用 WeakReference	下次 GC 必回收	ThreadLocal key、WeakHashMap
虚引用 PhantomReference	随时可回收，get 永远 null	跟踪对象回收（DirectByteBuffer 清理）

问：Safepoint 和 Safe Region 是什么？

答：GC 需要所有线程到达 **安全点** 才能开始（线程栈上没有对象引用在变化）。

Safepoint：线程主动轮询（方法调用、循环回边）检查是否需要暂停。

Safe Region：解决线程 sleep/block 时无法到达 Safepoint 的问题，进入安全区域时标记，离开时检查 GC 是否完成。

大厂追问

1. 对象一定在堆上分配吗？

不一定。逃逸分析后可能 **栈上分配** 或 **标量替换**。但默认说法「对象在堆上」指未优化的一般情况。

2. 为什么默认分代年龄是 15？

Mark Word 中 GC 年龄只占 4 bit，最大 15。-XX:MaxTenuringThreshold 可调整，但不超过 15。

3. G1 和 ZGC 怎么选？

G1：通用，JDK 9+ 默认，停顿可控（几十 ms）。ZGC：超大堆（>32G）、极低延迟要求（<1ms），JDK 21 分代后吞吐也提升。Parallel：批处理、吞吐优先。

4. 元空间会 GC 吗？

会。类加载器可回收时，其加载的类元信息可被 GC（Class Unloading），条件较苛刻（ClassLoader 和 Class 对象都不可达）。

5. dump 时为什么要加 live?

live 会先 Full GC 再 dump，减少无用对象干扰，聚焦存活对象。但可能丢失刚要回收的有用信息，排查泄漏时可不加 live。

6. 标记-清除算法的缺点?

产生 **内存碎片**，大对象可能找不到连续空间触发 Full GC；需要配合标记-整理或复制算法。

7. JIT 编译和解释执行的关系?

热点代码由 JIT 编译为本地机器码 (C1 快速编译 / C2 深度优化)，非热点仍解释执行。-XX:CompileThreshold 控制热点阈值。

8. 直接内存为什么不受 -Xmx 限制?

DirectByteBuffer 分配在堆外，受 -XX:MaxDirectMemorySize 限制，仍占物理内存，OOM 报 Direct buffer memory。